Programming Assignment 1: Algorithm Implementation Project

Patrick Rumble

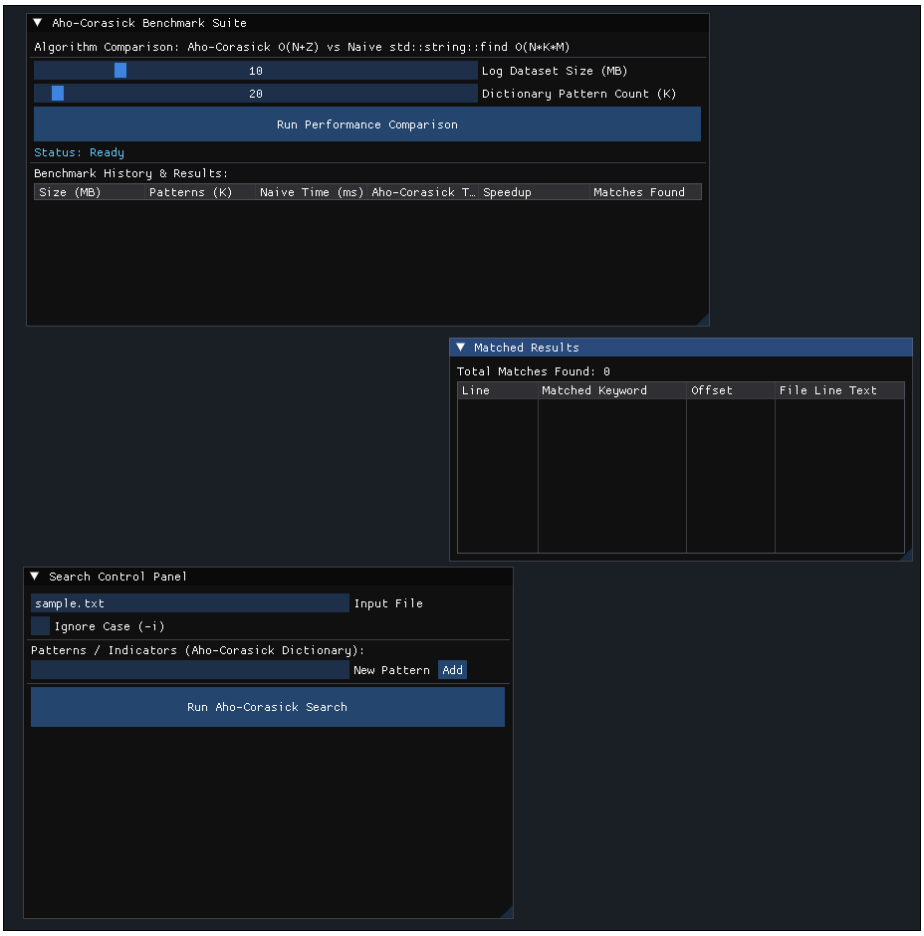
September 30, 2026

1 What I built

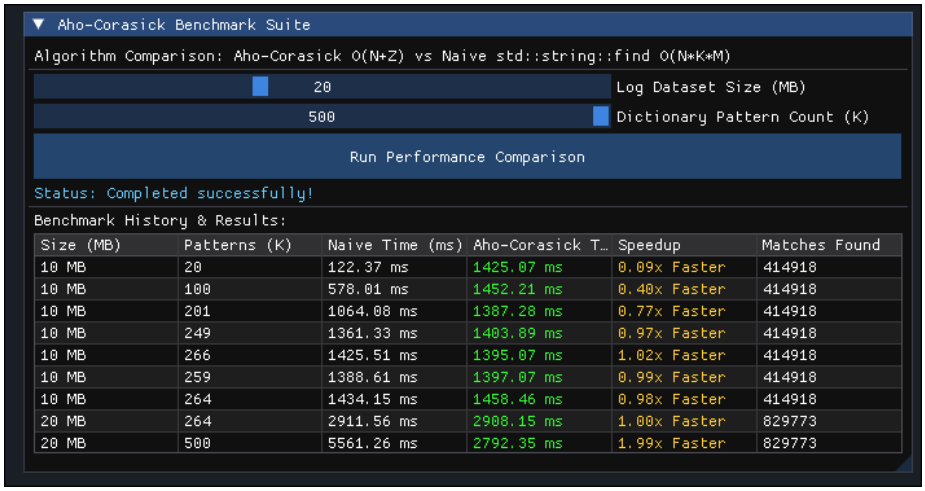
A simple GUI program using Aho-Corasick to match multiple patterns in a file, displaying the matches in a separate window. The program uses imgui, and takes an input file and patterns to filter.

2 The Track-Specific Writeup

The program has two main windows, and one optional window for benchmarking purposes. The user can input any number of patterns, an input file, and press the run button, and the results will show in the result window. The benchmark window (enabled at compile time) allows for simulation of any number of patterns used to find matches in a simulated dataset, and compares the speed with that of std::string::find.


Aho-Corasick Benchmark Suite
Algorithm Comparison: Aho-Corasick O(N+Z) vs Naive std::string::find O(N*K*M)
10
Log Dataset Size (MB)
20
Dictionary Pattern Count (K)
Run Performance Comparison
Status: Ready
Benchmark History & Results:
Size (MB)
Patterns (K)
Naive Time (ms)
Aho-Corasick T…
Speedup
Matches Found
Matched Results
Total Matches Found: 0
Line
Matched Keyword
Offset
File Line Text
Search Control Panel
sample.txt
Input File
Ignore Case (-i)
Patterns / Indicators (Aho-Corasick Dictionary):
New Pattern
Add
Run Aho-Corasick Search

As is visible in the attached image, std::string::find easily wins on time at low pattern counts, but as the number of patterns increases, the Aho-Corasick time stays consistent, eventually winning.



Size (MB)	Patterns (K)	Naive Time (ms)	Aho-Corasick T…	Speedup	Matches Found
10 MB	20	122.37 ms	1425.07 ms	0.09x Faster	414918
10 MB	100	578.01 ms	1452.21 ms	0.40x Faster	414918
10 MB	201	1064.08 ms	1387.28 ms	0.77x Faster	414918
10 MB	249	1361.33 ms	1403.89 ms	0.97x Faster	414918
10 MB	266	1425.51 ms	1395.07 ms	1.02x Faster	414918
10 MB	259	1388.61 ms	1397.07 ms	0.99x Faster	414918
10 MB	264	1434.15 ms	1458.46 ms	0.98x Faster	414918
20 MB	264	2911.56 ms	2908.15 ms	1.00x Faster	829773
20 MB	500	5561.26 ms	2792.35 ms	1.99x Faster	829773

The algorithm implementation sits inside the file ahocorasick.cpp. Code in this file is executed when the user clicks run, through the run_async_search function.

3 What I learned

Before working on this task I had not heard of Aho-Corasick, so everything about it was a learning experience. I was also highly suprised by how many patterns the Aho-Corasick implementation took to beat the std::string::find comparison.

4 AI Use

4.1 Which Tools I Used

For this task I primarily made use of Gemini's free tier, as this is one of the few AI tools I have used in the past.

4.2 What I Used Them For

Gemini was used to assist in writing the implementation, as well as provide feedback on ideas.

4.3 Examples of Where AI Was Unhelpful

Gemini produced some code with errors, for example when implementing the GUI, using ImVec2 instead of ImVec4 in its generated code, and leaving out

arguments in glClear(). Additionally, when attempting to add further functionality to the GUI, it just generated a non-functional textbox.

4.4 What I Understood and What I Took On Trust

I have placed a lot of trust in Gemini when it comes to this task. I have no experience with GUI programs or CMake, and was heavily reliant on Gemini for the code involved.

5 Video Walkthrough

https://youtu.be/Nwdvl02RuFU